ЛАБОРАТОРНА РОБОТА № 6

Тема: Віртуалізація

Мета: Отримати практичний досвід роботи щодо віртуалізації серверів та аплікацій.

Час виконання: 2 год.

ХІД РОБОТИ

Теоретичні відомості

Детально про віртуалізацію: типи, переваги та рішення

Віртуалізація набуває все більшої популярності, але мало хто по-справжньому розбирається, навіщо вона потрібна й які її принципи роботи. Почнемо ж ми з питання «що таке віртуалізація».

Що таке віртуалізація

Віртуалізація – це об’єднання різних обчислювальних ресурсів і їх подальше перенесення з фізичної машини на віртуальну. Так як ви використовуєте окремі RAM, CPU й інші обчислювальні ресурси віртуального сервера, є можливість розмістити відразу декілька самостійних віртуальних машин. Завдяки цьому ви:

- заощадите ресурси та бюджети на придбання додаткових обладнань;
- підвищите продуктивність IT-інфраструктури компанії.

Але віртуалізація не змогла б існувати, якби не було гіпервізора – програмного забезпечення, яке взаємодіє з ресурсами фізичного сервера. Іншими словами, гіпервізор необхідний, щоб віртуальні машини розпізнавали фізичні ресурси сервера та могли їх використовувати.

Типи віртуалізації

Існує 6 типів віртуалізації, які варіюються в залежності від використовуваного обладнання та додатків:

Віртуалізація операційних систем

1. Програмна віртуалізація

Вся віртуалізація проходить на рівні ядра операційної системи. В результаті кожна з окремих віртуальних машин працюють як самостійний сервер. Перевага програмної віртуалізації полягає в тому, що будь-які процеси можуть працювати на високій швидкості.

2. Апаратна віртуалізація

Подібна віртуалізація здійснюється на основі процесорної архітектури. Апаратна віртуалізація передбачає поділ процесора на моніторну і гостьову частини (root і non-root режими). При такій віртуалізації за допомогою гіпервізора можливо пряме управління гостьовими системами, які використовуються ізольовано один від одного.

3. Контейнерна віртуалізація

Контейнерна віртуалізація пов’язана з ізоляцією на рівні процесів ОС, доступна тільки в Linux і працює завдяки механізмам контейнеризації namespaces і cgroups. Таке рішення зазвичай використовується на рівні різних сервісів, які є частиною програми. Серед відомих прикладів: Docker і OpenVZ.

Віртуалізація програмного забезпечення

За допомогою віртуалізації програмного забезпечення створюється система з певним апаратним забезпеченням, що дозволяє відкривати гостьову ОС. Іншими словами, ви зможете відкрити операційну систему Android навіть на базі Microsoft Windows, користуючись ресурсами обладнання.

Віртуалізація програмного забезпечення включає в себе 2 підтипи:

- Віртуалізація додатків
- Віртуалізація процесів і сервісів

Віртуалізація інфраструктури

1. Віртуалізація серверів

Мета подібної віртуалізації – це створення архітектури програмного забезпечення, яке буде підтримувати роботу декількох ОС на базі одного фізичного сервера. Програмне забезпечення кожного сервера незалежно працює від фізичних пристроїв. Подібна віртуалізація управляється гіпервізором і не поступається в ефективності класичного варіанту.

2. Віртуалізація десктопів

Досягнення віртуалізації допомагають віртуалізувати окремий десктоп кожного співробітника та надати до роботи тільки термінальну частину.

3. Віртуалізація додатків

Подібне рішення дозволяє відкривати додатки на пристроях без необхідності встановлювати їх на тих же комп’ютерах. Віртуалізація додатків означає розгортання віртуальної середовища для додатків файлової системи, реєстру й API операційної системи без втручання в роботу ОС.

Віртуалізація віддалених робочих столів

Щоб співробітники могли працювати поза офісом, їх віддалені робочі столи розміщуються на віртуальному сервері. Співробітники можуть працювати з будь-якого пристрою, місця і в будь-який зручний час. Віддалені робочі столи захищаються спеціальними протоколами безпеки, які мінімізують ризик кібератак або витоку даних.

Віртуалізація систем зберігання даних

За допомогою віртуалізації систем зберігання даних (СЗД) компанії можуть рівномірно розподіляти навантаження з необхідними параметрами, такі як ефективність, доступність, захист і відмовостійкість. Така віртуалізація допоможе:

- спростити перенесення корпоративних даних;
- убезпечити зберігання даних;
- оптимізувати місце на серверах;
- заощадити на масштабування IT-інфраструктури.

Віртуалізація мережі

Віртуалізація мережі необхідна для розгортання кількох підмереж. Організація самостійно вирішує, чи буде між ними зв’язок, щоб контролювати переміщення даних і таким чином підвищити рівень кіберзахисту. Крім цього, віртуалізація мережі зручна для моніторингу та визначення використання корпоративної інформації. В результаті компанія отримує:

- масштабована мережа;
- спрощений аналіз даних;
- високий рівень безпеки.

Віртуалізація серверів

Серед усіх типів віртуалізації особливо виділяється серверна віртуалізація. Вище ми коротко описали визначення, тепер давайте розглянемо головні переваги, причини та рішення для віртуалізації серверів.
Переваги серверної віртуалізації

- Оптимізація використання ресурсів зберігання і обчислення. Організації більше не треба виділяти приміщення для зберігання фізичного обладнання. Крім цього, всі ресурси віртуальної машини використовуються з максимальною продуктивністю на відміну від фізичних аналогів, які можуть частково простоювати.
- Контроль кількості фізичних серверів. Замість великої кількості фізичних серверів досить буде кілька потужних фізичних машин з безліччю віртуальних середовищ.
- Міграція корпоративної інформації. Будь-яке перенесення даних з однієї віртуальної машини на другу проходить швидко і непомітно для співробітників.
- Продуктивне прикладне ПЗ. Раніше додатки могли споживати усі ресурси фізичного обладнання, але з серверною віртуалізацією навантаження може автоматично розподілятися на інші віртуальні середовища.
- Доступність і безперервність роботи. Адже швидкість запуску віртуальних серверів набагато вища, а резервне копіювання робиться швидко та зручно.
- Контроль віртуального середовища. Компанії знадобиться менше фахівців для адміністрування віртуальних серверів. Як альтернатива, організація може скористатися послугами провайдера, який спеціалізується на управлінні віртуальним середовищем.

Рішення для віртуалізації серверів

Розібравши поняття віртуалізації і її основні типи, тепер давайте розглянемо популярні системи віртуалізації серверів, у яких є свої відмінні риси і переваги. Серед них:
Система віртуалізації Microsoft Hyper-V – це професійне рішення з апаратної віртуалізації, за допомогою якого можна створити декілька самостійних програмних середовищ на одному фізичному сервері. В організації з’являється можливість взаємодіяти з апаратним обладнанням через платформу віртуалізації Hyper-V. У більшості випадків система віртуалізації Microsoft Hyper-V використовується в якості:

- рішення для віртуалізації робочих місць співробітників компанії;
- рішення для розміщення та утримання серверів в ЦОД;
- інструменту девелоперів для безпечного тестування нових функцій ПО перед їх впровадженням.

Система віртуалізації VMware ESXI – це апаратний гіпервізор, який націлений на віртуалізацію продуктів в центр обробки даних і створення хмарної IT-інфраструктури. Ця платформа віртуалізації працює з різними ОС і в основному використовується для:

- консолідації роботи серверів;

- підвищення ефективності роботи і конкурентоспроможності;
- оптимізації адміністрування IT-інфраструктури через гіпервізор;
- скорочення капітальних і операційних витрат;
- забезпечення безперервності роботи бізнесу;
- забезпечення повноцінного аварійного відновлення.

Переваги віртуалізації

- Продуктивність
  За допомогою віртуалізації ви можете розділити і використовувати апаратні ресурси однієї фізичної машини на декількох віртуальних. Корпоративні системи можуть також повноцінно працювати навіть через віртуальне середовище. Це економить витрати на придбання та підтримку апаратного обладнання.
- Безпека
  Кластеризація віртуалізованих серверів гарантує захищене побудова відмовостійкої системи. Ви також можете скористатися функцією бекапа об’єктів і даних на віртуалізованих серверах У разі атаки система швидко відновить свої процеси і сервіси.
- Налагодження процесів
  Попередні дві переваги сприяють створенню ефективної стратегії відновлення та перевірки служб і процесів перед їх конкретним впровадженням. Так зменшується кількість помилок в роботі і час на їх рішення.
- Ізоляція між різними віртуальними серверами
  Поділ однієї фізичної машини на кілька віртуальних означає, що компанія мінімізує кількість збоїв, які могли б виникнути під час спільної роботи фізичних серверів.
- Рівномірний розподіл ресурсів
  Віртуалізація допоможе навести порядок в IT-інфраструктурі. Ви самостійно визначте кількість необхідних ресурсів і розставити пріоритети їх використання на віртуальних машинах Більш того, віртуалізація рівномірно розподіляє весь трафік по віртуальним серверах.
- Високий рівень доступності
  У ваших співробітників буде доступ до віддалених робочих столів незалежно від локації і пристрої. При цьому не постраждає продуктивність роботи.

Контейнери

Що таке Docker: для чого він потрібен і де використовується

Docker — один із найвідоміших інструментів для роботи з контейнерами. У статті розповімо, що таке контейнери, де вони застосовуються і чим можуть бути корисні.

В кінці буде практична частина: ми створимо невелику програму, обернемо її в образ і запустимо. Всі дії показуватимемо на прикладі віртуальної машини.

Контейнери – гарна альтернатива апаратній віртуалізації. Вони дозволяють запускати додатки в ізольованому оточенні, але споживають набагато менше ресурсів.

В першу чергу ця стаття буде корисна тим, хто взагалі не знайомий з контейнерами або Docker. Ми розповімо самі базові речі, а наш приклад створення додатка буде досить простим. Але це дозволить вам зрозуміти основи Docker і далі рухатися — вивчати складніші матеріали.

Що таке контейнери

Перш ніж розповідати про Docker, слід сказати кілька слів про технологію контейнеризації.

Контейнери — це спосіб стандартизації розгортки програми та відокремлення її від загальної інфраструктури. Примірник програми запускається в ізольованому середовищі, що не впливає на основну операційну систему.

Розробникам не потрібно замислюватися, в якому оточенні працюватиме їхня програма, чи будуть там потрібні налаштування та залежності. Вони просто створюють додаток, упаковують усі залежності та налаштування у деякий єдиний образ. Потім цей образ можна запускати на інших системах, не турбуючись, що програма не запуститься.

Docker – це платформа для розробки, доставки та запуску контейнерних програм. Docker дозволяє створювати контейнери, автоматизувати їх запуск та розгортання, керує життєвим циклом. Він дозволяє запускати безліч контейнерів на одній машині.

Контейнеризація схожа на віртуалізацію, але це не одне й те саме. Віртуалізація запускає повноцінний хост на гіпервізорі зі своїм віртуальним обладнанням та операційною системою. При цьому всередині однієї ОС можна запустити іншу ОС. У разі контейнеризації процес запускається прямо з ядра основної операційної системи та не віртуалізує обладнання. Це означає, що контейнеризоване додаток може працювати тільки в тій самій ОС, що й основна. Контейнери не віртуалізують обладнання, тому споживають менше ресурсів.

Переваги використання контейнерів Docker

Контейнери спрощують роботу як програмістам, і адміністраторам, які розгортають ці додатки.

Docker вирішує проблеми залежностей та робочого оточення

Контейнери дозволяють запакувати в єдиний образ програму та всі її залежності: бібліотеки, системні утиліти та файли налаштування. Це полегшує перенесення програми на іншу інфраструктуру.

Наприклад, розробники створюють додаток у системі розробки - там все налаштовано, програма працює. Коли воно готове, його потрібно перенести в систему тестування, а потім у продуктивне середовище. Якщо в одній з них немає потрібної залежності, програма не працюватиме. Програмістам доведеться відволіктися від розробки та спільно з командою підтримки розібратися в ситуації.

У контейнерах такої проблеми немає, оскільки вони містять все необхідне для запуску програми. Фахівці займаються розробкою, а не розв'язанням інфраструктурних проблем.

Ізоляція та безпека
Контейнер — це набір процесів ізольованих від основної операційної системи. Програми працюють лише всередині контейнерів і не мають доступу до основної операційної системи. Це підвищує безпеку додатків: вони не зможуть випадково чи навмисне нашкодити основній системі. Якщо програма в контейнері завершиться з помилкою або зависне, це ніяк не торкнеться основної ОС.

Прискорення та автоматизація розгортання додатків та масштабованість
Контейнери полегшують розгортання додатків. У класичному підході для встановлення програми потрібно зробити кілька дій: виконати скрипт, змінити налаштування файлів і так далі. У цьому процесі не виключена можливість людської помилки: користувач запустить скрипт двічі, переплутає послідовність або щось не зрозуміє. Контейнери дозволяють повністю автоматизувати цей процес, оскільки включають всі необхідні залежності і порядок виконання дій.

Також контейнери спрощують розгортання на кількох серверах. У класичному підході для того, щоб розгорнути один і той же додаток на декількох машинах, потрібно буде повторювати одні й ті самі дії. Контейнери позбавляють цієї рутинної роботи і дозволяють автоматизувати розгортання.

Контейнери наближають до мікросервісної архітектури
Контейнери добре вписуються у мікросервісну архітектуру. Це підхід до розробки, у якому додаток розбивається на невеликі компоненти, наскільки можна незалежні. Зазвичай протиставляється монолітної архітектури, де всі частини системи сильно пов'язані одна з одною.

Це дозволяє розробляти нову функціональність швидше, адже у випадку з монолітною архітектурою зміна якоїсь частини може торкнутися всієї іншої системи.

Docker compose - одночасно розгорнути кілька контейнерів
Docker-compose дозволяє розгортати та налаштовувати кілька контейнерів одночасно. Наприклад, для веб-програми потрібно розгорнути стек LAMP: Linux + Apache, MySQL, PHP. Кожна з програм — це окремий контейнер для ОС Linux. Але в цій ситуації нам потрібні саме всі контейнери разом, а не окремо взята програма. Docker-compose дозволяє розгорнути та налаштувати всі програми однією командою, а без нього довелося б розгортати та налаштовувати кожен контейнер окремо.

Зберігання даних у Docker
Однією з головних особливостей контейнерів є ефемерність. Це означає, що контейнери можуть бути зупинені, перезапущені або знищені. При цьому всі накопичені дані в контейнері будуть втрачені. Тому програми потрібно розробляти так, щоб вони не покладалися на сховище даних у контейнері, це називається принципом Stateless.

Це добре підходить для програм або сервісів, які не зберігають результати своєї роботи. Наприклад, функції розрахунку або перетворення даних: їм на вхід надійшов один набір даних, вони його перетворили або розрахували та повернули результат. Все нічого нікуди зберігати не потрібно.

Але далеко не всі програми такі, і є багато даних, які потрібно зберегти. У контейнерах цього передбачено кілька способів.

Тома (Docker volumes)
Це спосіб, коли Docker сам створює директорії для зберігання даних. Їх можна зробити доступними для різних контейнерів, щоб вони могли обмінюватись даними. За замовчуванням ці директорії створюються на хост-машині, але можна використовувати віддалені сховища: файловий сервер або об'єктне сховище.

Монтування каталогу (bind mount)
У цьому випадку директорія спочатку створюється на хост-машині, а вже потім монтується в контейнери.

Але цей спосіб не рекомендується, тому що він ускладнює резервне копіювання, міграцію та спільне використання даних кількома контейнерами.

Архітектура (компоненти) Docker
Тепер докладніше про компоненти, з яких складається Docker.

Docker daemon
Це якийсь резидентний процес, який запущений на хост-машині постійно. Він володіє всією інфраструктурою, а також надає інтерфейс взаємодії з контейнерами, що включає створення та видалення, запуск та зупинку.

У ранніх версіях платформи Docker можна зустріти згадку про dockerd, але зараз демони вже встигли розбитися на окремі проекти. Все частіше можна зустріти його сучасника - containerd.

Docker client (клієнт)
Це інтерфейс командного рядка для керування Docker Daemon. Ми користуємося цим клієнтом, коли створюємо та розвертаємо контейнери, а клієнт надсилає ці запити в Docker daemon.

Docker image (образ)
Це незмінний файл (образ), з якого розгортаються контейнери. Програми упаковуються саме в образи, з яких потім вже створюються контейнери. У технічній літературі можна також зустріти опис зображення як шаблону запуску процесу.

Наведемо аналогію з прикладу установки операційної системи. У дистрибутиві (образі) ОС є все, що потрібно для її встановлення. Але цей образ не можна запустити, спочатку його потрібно «розгорнути» в готову ОС. Так ось, дистрибутив для установки ОС – це образ, а встановлена та працююча ОС – це контейнер. Але контейнери зазвичай розвертаються однією командою - це набагато простіше і швидше, ніж встановлення ОС.

Docker container (контейнер)
Це вже розгорнуте з образу і додаток, що працює.

Docker Registry
Це репозиторій із образами. Розробники створюють образи своїх програм і викладають їх у репозиторій, щоб їх можна було завантажити та скористатися ними. Поширений репозиторій - Docker Hub. У ньому зібрані образи багатьох популярних програм чи платформ: бази даних, веб-сервери, компілятори, операційні системи тощо. Також можна створити свій приватний репозиторій, наприклад, в середині компанії. Розробники розміщуватимуть там образи, які використовуватимуться всією компанією.

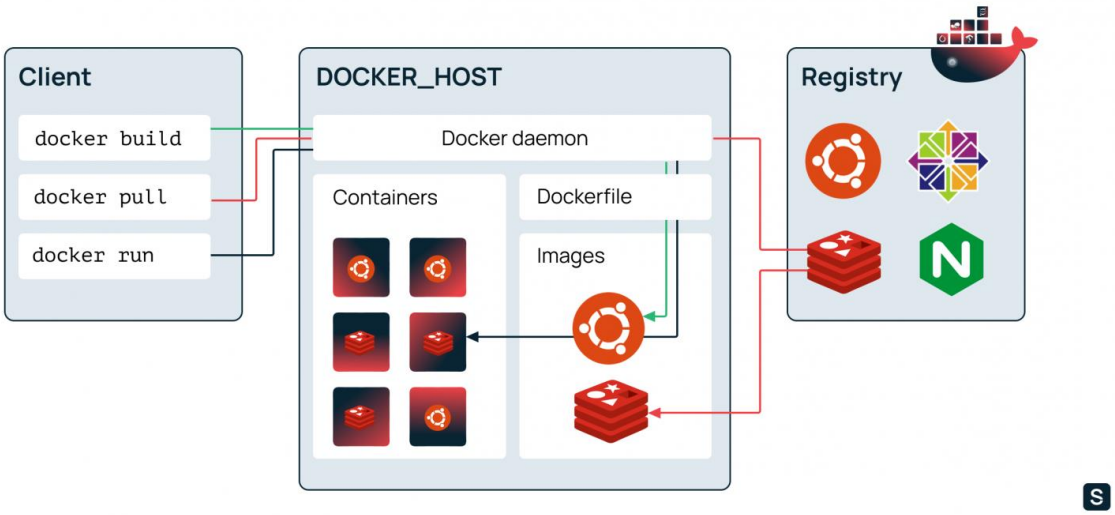
Dockerfile
Dockerfile – це інструкція для збирання образу. Це простий текстовий файл, що містить одну команду в кожному рядку. У ньому вказуються всі програми, залежності та образи, які потрібні для розгортання образу.

Для прикладу розглянемо Dockerfile, який ми будемо використовувати далі, щоб розгорнути власну програму:

FROM python:3
COPY main.py /
CMD ["python", "./main.py"]

Перший рядок означає, що за основу ми беремо образ під назвою python версії 3 це називається базовий образ. Docker знайде його в docker registry, завантажить і використовуватиме за основу. Другий рядок означає, що потрібно скопіювати файл main.py в корінь файлової системи контейнера. Третій рядок означає, що потрібно запустити python і передати йому як параметр назву файлу main.py.

Далі розглянемо приклади кількох команд докеру і що відбувається, коли ми їх виконуємо.



Всі ці команди виконуються в Docker client, який відправляє їх у Docker daemon:

Команда **docker build** (зелена стрілка) читає dockerfile та збирає образ.
Команда **docker pull** (червона стрілка) завантажує образ із docker registry. За умовчанням docker викачує образи з публічного репозиторію Docker Hub. Але можна створити свій репозиторій та налаштувати докер, щоб він працював з ним.
Команда **docker run** (чорна стрілка) бере образ і запускає контейнер.

Створюємо віртуальну машину для роботи, або використовуємо існуючу

Установка Docker

Ми розглянемо встановлення докеру на прикладі Ubuntu. Якщо у вас інший дистрибутив Linux або операційна система, шукайте відповідну інструкцію на офіційному сайті.

Для початку синхронізуємо пакетну базу apt і встановимо необхідні залежності:

```
sudo apt-get update

sudo apt-get install \
    apt-transport-https \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
```

Далі імпортуємо GPG-ключ для репозиторію docker:

```
curl-fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

Тепер додамо новий репозиторій до списку apt:

```
echo \
  "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Тепер можна встановлювати докер:

```
sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli containerd.io
```

За замовчуванням доступ до docker daemon є тільки у користувача root. Щоб з докером могли працювати інші користувачі, їх потрібно додати в спеціальну групу — docker. Виконайте цю команду з-під звичайного користувача:

```
sudo usermod -aG docker $USER
```

Після цього необхідно перелогінитись, щоб зміна набула чинності.

Запуск контейнера

Тепер спробуємо запустити якийсь готовий додаток. Виконайте команду:

docker run ubuntu echo 'hello from ubuntu'

Команда docker run створює та запускає контейнер із образу. У цьому прикладі ми створюємо контейнер з образу ubuntu, потім виконуємо команду echo 'hello from ubuntu'. Але так як у нас чиста установка докера і ми не завантажили жодного образу, докер спочатку знайде цей образ у публічному репозиторії Docker Hub, завантажить, а потім створить контейнер. Наступного разу, коли нам знадобиться образ ubuntu, докер вже не завантажуватиме його.

Після виконання команди в терміналі з'явиться рядок hello від ubuntu, і контейнер відразу зупиниться. Тепер виконаємо іншу команду:

docker run -it ubuntu

Ця команда запустить контейнер в інтерактивному режимі, тобто контейнер запуститься і чекатиме на подальші команди. При цьому ми опинимося всередині операційної системи контейнера: запуститься оболонка (bash) і ми зможемо виконувати якісь команди всередині контейнера. Щоб вийти з контейнера, введіть exit.

Створення власного образу та запуск контейнера
Тепер створимо HelloWorld-додаток на Python, обернемо його в образ і запустимо.

Для початку створимо директорію, в якій ми працюватимемо і перейдемо до неї:

mkdir first-docker-app
cd first-docker-app

Створимо файл main.py і запишемо в нього один рядок коду:

echo 'print("Hello from python");' >> main.py

Перевіримо, що наша програма працює. Для цього виконаємо команду:

python main.py

У консолі має вийти повідомлення Hello from python. Це і є наш простий додаток. Тепер потрібно обернути його в докер-образ. Для цього створимо файл Dockerfile і напишемо в ньому три рядки:

FROM python:3
COPY main.py /
CMD ["python", "./main.py"]

У першому рядку ми вказуємо образ, який беремо за основу. Так як ми пишемо додаток на Python, потрібно, щоб у нашому образі він уже був встановлений. Найпростіший спосіб це зробити використовувати готовий офіційний образ з Docker Hub. Цифра 3 це тег. Це означає, що потрібно використовувати третю версію Python. Натомість можна було б використовувати тег latest, який означає останню версію, або можна було вказати номер конкретної версії, наприклад 3.8.8.

У другому рядку ми копіюємо наш файл main.py до кореневої директорії образу.

Третій рядок - запускаємо python і передаємо йому як параметр ім'я нашого файлу.

Тепер із цього докер-файлу можна збирати образ. Виконаємо команду:

docker build -t first-docker-app .

Параметр -t означає ім'я нашого образу, ми назвали його first-docker-app. (крапка в кінці попередньої команди є обовязкова)

Так як у нас ще немає завантаженого python образу, то докер сам скачає його з Docker Hub і потім буде використовувати його як основу для створення нашого образу.

Перевіримо список встановлених у нас образів:

docker images

Ми побачимо, що в нас встановлено три образи:

```
REPOSITORY          TAG       IMAGE ID       CREATED          SIZE
first-docker-app    latest    649cceb4dfd2   4 seconds ago    885MB
python              3         b1aa63f57d3c   2 days ago       885MB
ubuntu              latest    8e428cff54c8   4 days ago       72.9MB
```

first-docker-app – це наш образ, який ми щойно створили. python - це образ python, який докер автоматично завантажив, щоб зібрати наш образ. ubuntu – образ, який ми пробували для запуску готової програми.

Тепер створимо контейнер із нашого образу та запустимо його:

docker run first-docker-app

В результаті нам виведеться результат: Hello from python.

Підсумок: Ми створили свою програму, запакували її в докер-образ і запустили. Звісно, це дуже простий приклад. Наша програма складається всього з одного рядка, а dockerfile з трьох. Але це дозволяє зрозуміти базові принципи роботи докера, як він влаштований, як створювати свої образи та запускати контейнери.

Список корисних команд

Тепер наведемо список корисних команд, які можуть стати в нагоді при роботі з докером.

Переглянути список усіх контейнерів
Ця команда виведе список усіх докерів контейнерів:

docker ps

Але за замовчуванням виводяться лише працюючі контейнери. Щоб вивести всі, в тому числі зупинені, використовуйте опцію -a:

docker ps -a

Зупинити та видалити всі докер контейнери
Щоб видалити контейнери, їх потрібно спочатку зупинити. Перша команда зупинить запущені контейнери, якщо вони є. А друга команда – видалить їх.

docker stop $(docker ps -a -q)
docker rm $(docker ps -a -q)

Запустити контейнер із наступним видаленням
За замовчуванням контейнери зупиняються після завершення роботи, але не видаляються. Вони зберігають свій стан і за необхідності їх можна знову запустити. Щоб контейнер видалявся після зупинки, додайте до команди docker run параметр —rm, наприклад:

docker run --rm ubuntu echo 'hello from ubuntu'
Переглянути список усіх скачаних образів

docker images

Видалити докер образ

docker rmi <ім'я-образу>

Якщо цей образ має контейнери, нехай навіть зупинені, докер не дозволить його видалити. Він видасть повідомлення:

unable to remove repository reference <ім'я-образу> (must force) - container <id-контейнера> is using its
Щоб примусово видалити образ, додайте -f:

docker rmi -f <ім'я-образу>

Отримати список усіх контейнерів, створених із певного образу

docker ps -a --filter ancestor=<назва-образу>

Підсумок
Ми розглянули, що таке контейнери та Docker, як вони працюють і чим відрізняються від віртуалізації. Також ми створили простий python-додаток, обернули його в образ та запустили контейнер.

Я розповів основи технологій, але не торкнулися складніших тем, на кшталт Docker Swarm, налаштування мережі або налаштування процесів CI/CD. Але цього цілком достатньо, щоб поринути в основи технологій.

Завдання для виконання:

1. Встановити Docker.
2. За допомогою docker compose розгорнути wordpress в контейнері.
3. За допомогою Dockerfile зробити свій власний контейнер з nginx на базі дистрибутива debian.
4. Оформити звіт про виконання роботи зі знімками екрана (або його частини), які ілюструють виконання завдань.